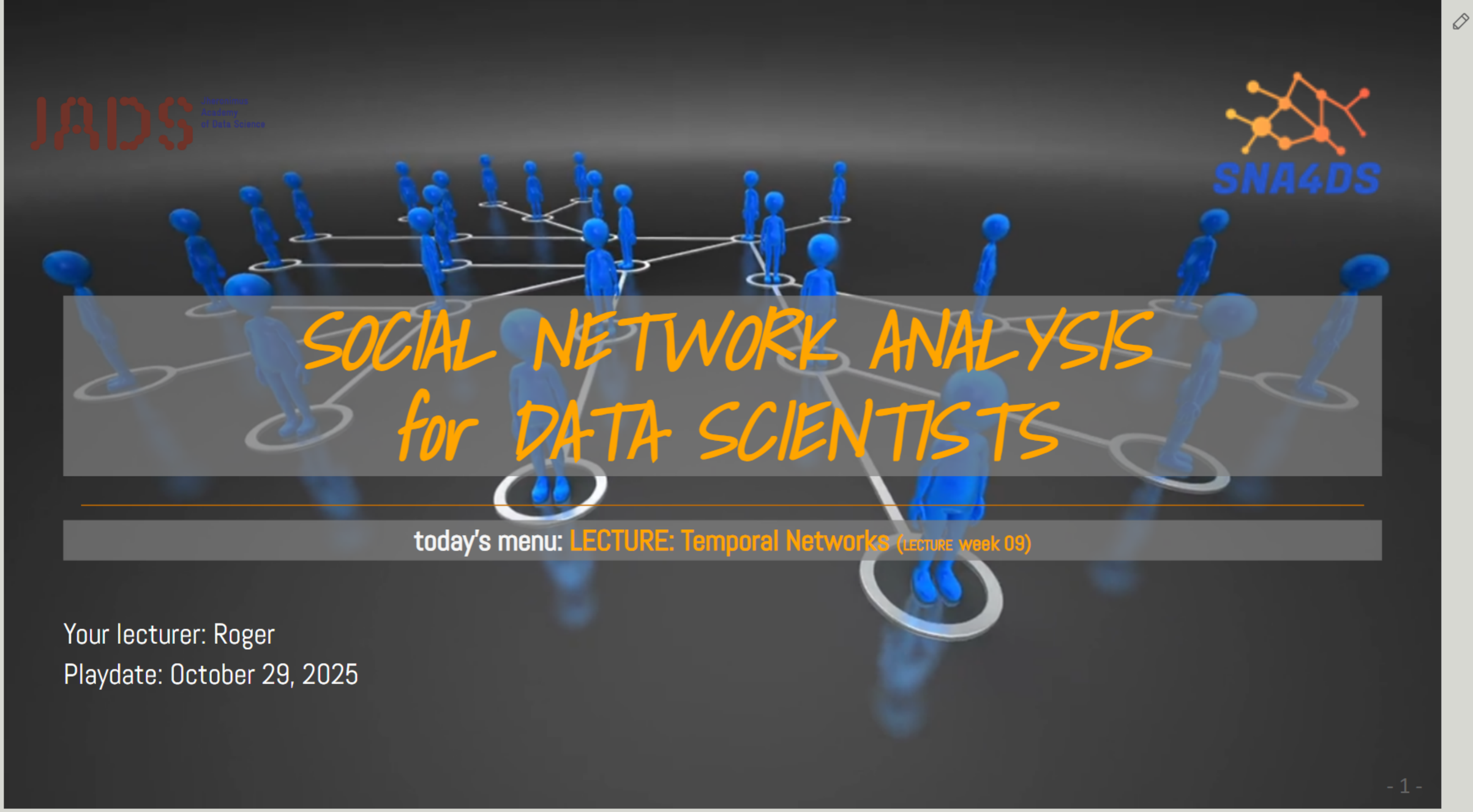


SOCIAL NETWORK ANALYSIS for DATA SCIENTISTS

today's menu: LECTURE: Temporal Networks (LECTURE week 09)

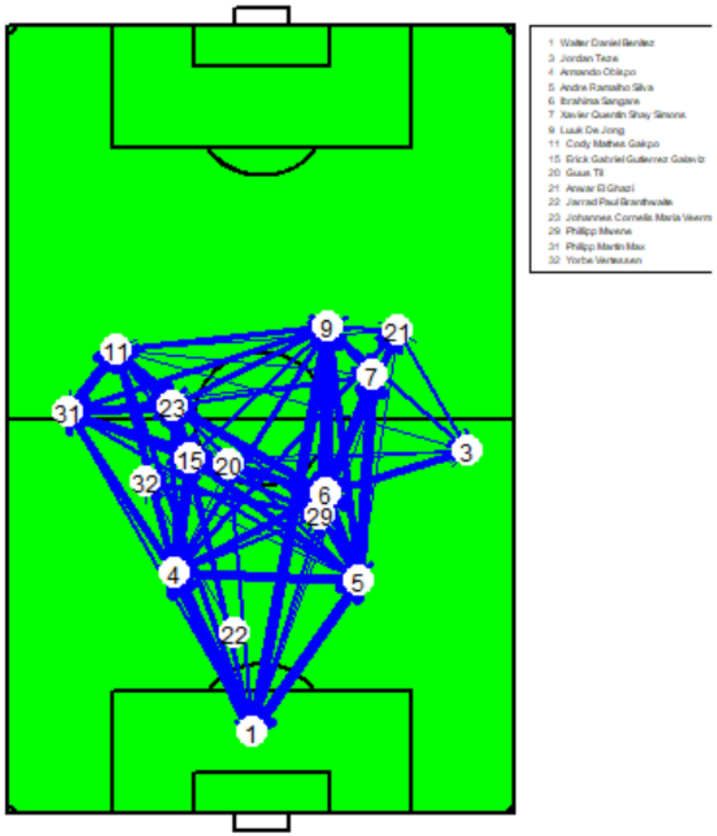
Your lecturer: Roger

Playdate: October 29, 2025

Ball passing network in soccer

Ajax - PSV = 1 - 2 (Nov. 2022)

Surprising observations



Some unexpected observations:

- no edge between 23 (Veerman) and 20 (Til)

although their geographic distance is low.

How can this be explained?

Other examples of temporal networks?

Characteristics of temporal networks

- edges have a start and an end (or they are instantaneous)
- edge attributes may vary over time
- network composition can change (ie. vertices join and leave the network)
- vertex attributes may vary over time
- (strengths of) effects may change over time, or be a function of time (e.g., time of day)

How do you analyze temporal networks?

SNA4DS

I: Collapse the network into a static (valued) network

- OK if the network is quite stable & if the temporal order is (deemed) irrelevant

➔ You can now use all of the tools you learned in this course.

How do you analyze temporal networks?

SNA4DS

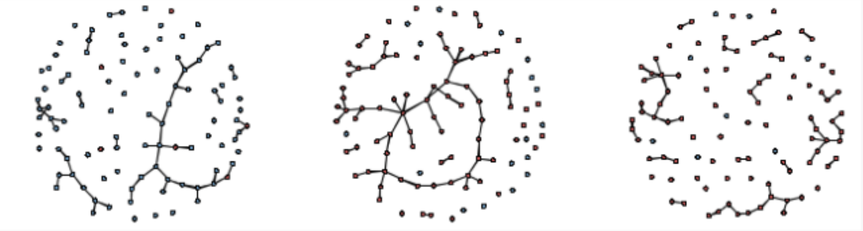
II: Collapse the network into temporal slices

Then analyze each slices as a static network.

- OK if the slices are meaningful by themselves & if enough data per slice

Note that the width of time slices is often arbitrary and a result of how you collected the data.

➔ You can now use all of the tools you learned in this course *on each slice* separately.



Some events:

23" 0 - 1 (#9, de Jong)

50" 0 - 2 (#17, Gutierrez)

62" In: #3 (El Ghazi), out: #23 (Veerman)

62" In: #21 (Teze), out: #31 (Max)

81" In: #32 (Vertessen), out: #9 (de Jong)

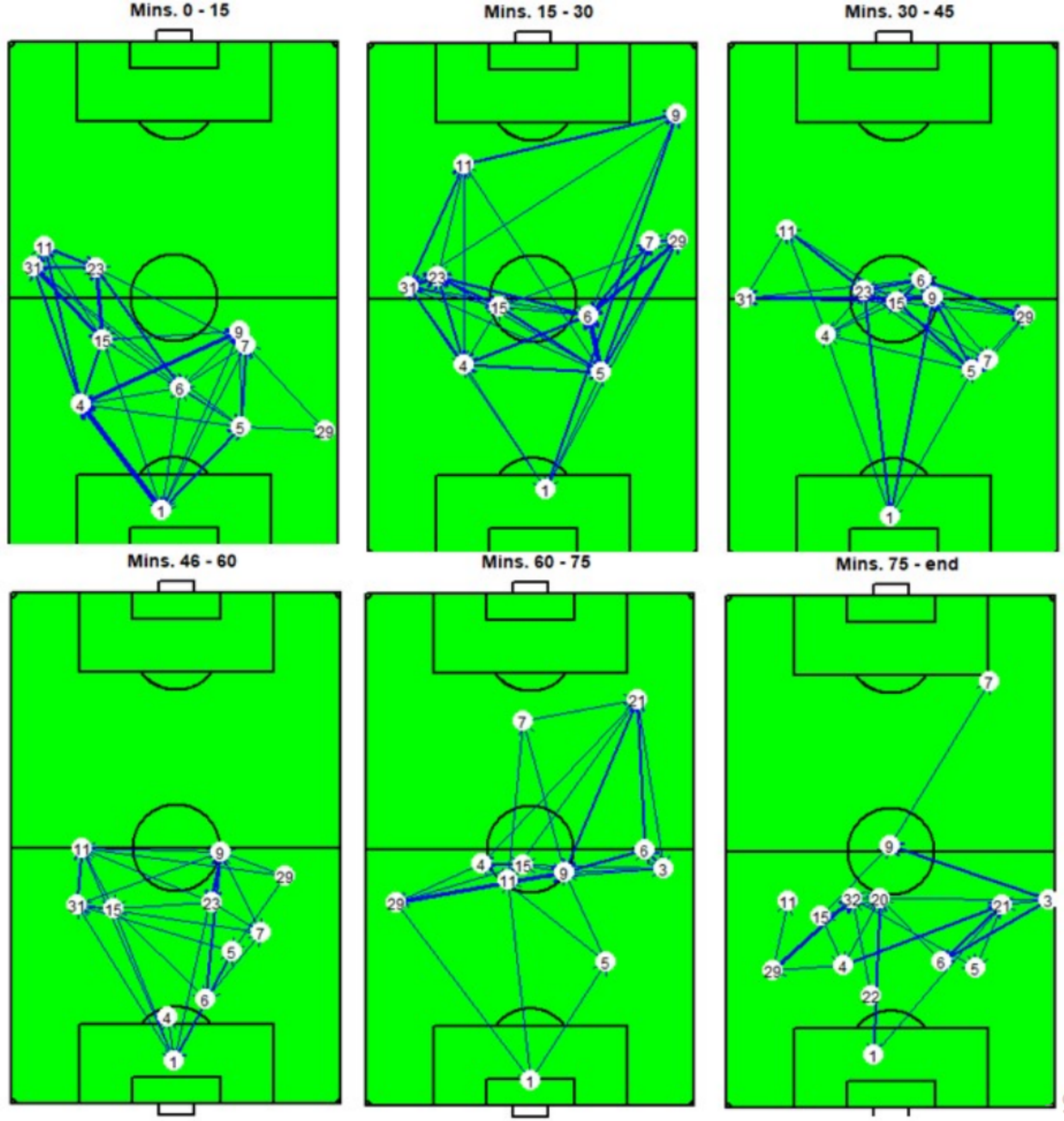
81" In: #20 (Til), out: #7 (Simons)

83" 1 - 2 (#18, Luca)

87" In: #22 (Branthwaite), out: #11 (Gakpo)

The networks are different for each quarter. Often, soccer analysts analyze the networks per quarter using the desciptive tools discussed in this course.

(in some plots, you only see 10 players. This happens due to overplotting, when two players occupied almost exactly the same position)



How do you analyze temporal networks?

SNA4DS

III: Do not collapse: take the order of events into account

You discard the exact timing of the interactions, but you focus on what happens first, what happens next, et cetera.

- helps to see ordering of interactions
- allows to uncover temporal motifs

➔ The statistical models for this are surprisingly straightforward

How do you analyze temporal networks?

SNA4DS

IV: Take the full time stamps of edges into account

This is the most informative approach.

However, it requires quite advanced statistical methods.

- helps to study exactly "how long it takes before..."
- allows for extremely informative and detailed models and hypothesis testing
- methods are still very much in development (JADS & TiU are world-leading in this field)

Data structure

The network slice that led to the 0 - 1 goal by Luuk de Jong

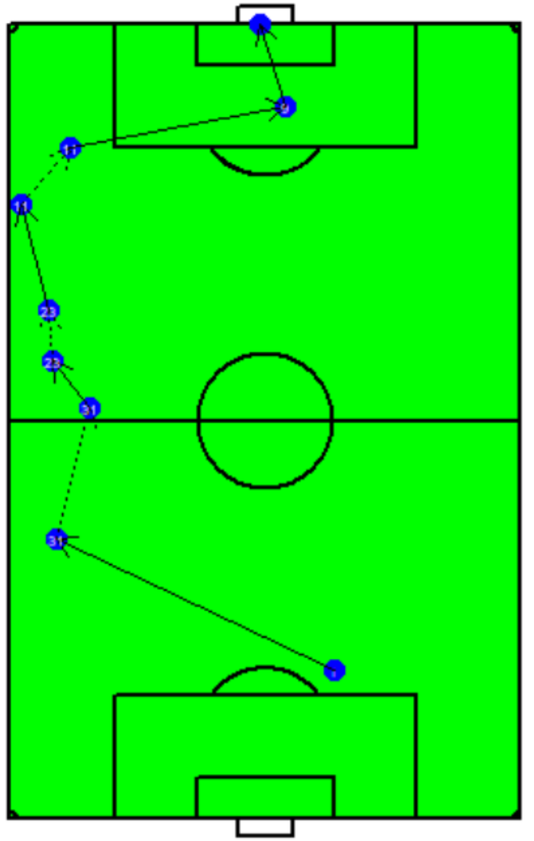
	onset	terminus	tail	head	duration
1	1318.04	1320.92	Benitez (#1)	Max (#31)	2.88
2	1324.58	1325.60	Max (#31)	Veerman (#23)	1.02
3	1327.84	1328.74	Veerman (#23)	Gakpo (#11)	0.90
4	1332.34	1332.43	Gakpo (#11)	De Jong (#9)	0.09
5	1333.65	1334.00	De Jong (#9)	Goal	0.35

In this network, it is useful to add vertex **geographic location** and **edge type** (e.g., *which foot*, *success*, *speed*, *under pressure*, etc.).

The full passing path took 5.24 seconds (from start to goal scored).

Edges have start and end times. So can vertices.

Note: in this example, there are also self-edges/loops and "non-player" edges (e.g., goal, line, woodwork)



Analysis of temporal network data

- repeated observations of a network
  - TERGM (Temporal ERGM): the focus of this lecture
- network data with starting and ending times of edges and vertices
  - Descriptive and exploratory analysis: the focus of the tutorial
  - Statistical models: beyond this course
  - Deep learning models: lecture in two weeks

Analyzing a series of networks: TERGM's

The ERGM (for a single network) is as follows:

$$P(N,\theta) = \frac{exp(\theta' h(N))}{c(\theta)}$$

If a network depends on K networks before it, then

$$P(N^t | N^{t-K}, \ldots, N^k, \theta) = \frac{exp(\theta' h(N^t, N^{t-1}, \ldots, N^{t-K}))}{c(\theta, N^t, N^{t-1}, \ldots, N^{t-1})}$$

So: you model all networks together, where network N^t depends on the K previous networks

This is the *TERGM* (very similar to the ERGM, as you can see)

In fact: it is so similar, that you can just think of it as an ERGM and apply what you already learnt.

The `btergm` package

You experienced the speed of the ERGM....it is slooooooooooow....because it uses MCMC-LE.

Now imagine running an ERGM model with 5, 10, 20, or 50 networks that depend on each other.
How fast would that run?

This is why we prefer to use MPLE instead of MCMC-LE. MPLE is much, much, much faster than MCMC-LE.
But because it underestimates uncertainty in the model, the standard errors are too small.

This can largely be corrected with **bootstrapping**. This slows the goodness-of-fit calculations down, but that is OK since MPLE is fast for the rest.

This is implemented in the `btergm` package.

The `btergm` package

The `btergm` package is fully compatible with the `ergm` package:

- you can use the same terms
- you can use the same network objects (but now a *list* of networks, instead of a single network)

Example: International alliances

```
data("alliances", package = "SNA4DSData")
class(allianceNets)
```

```
[1] "list"
```

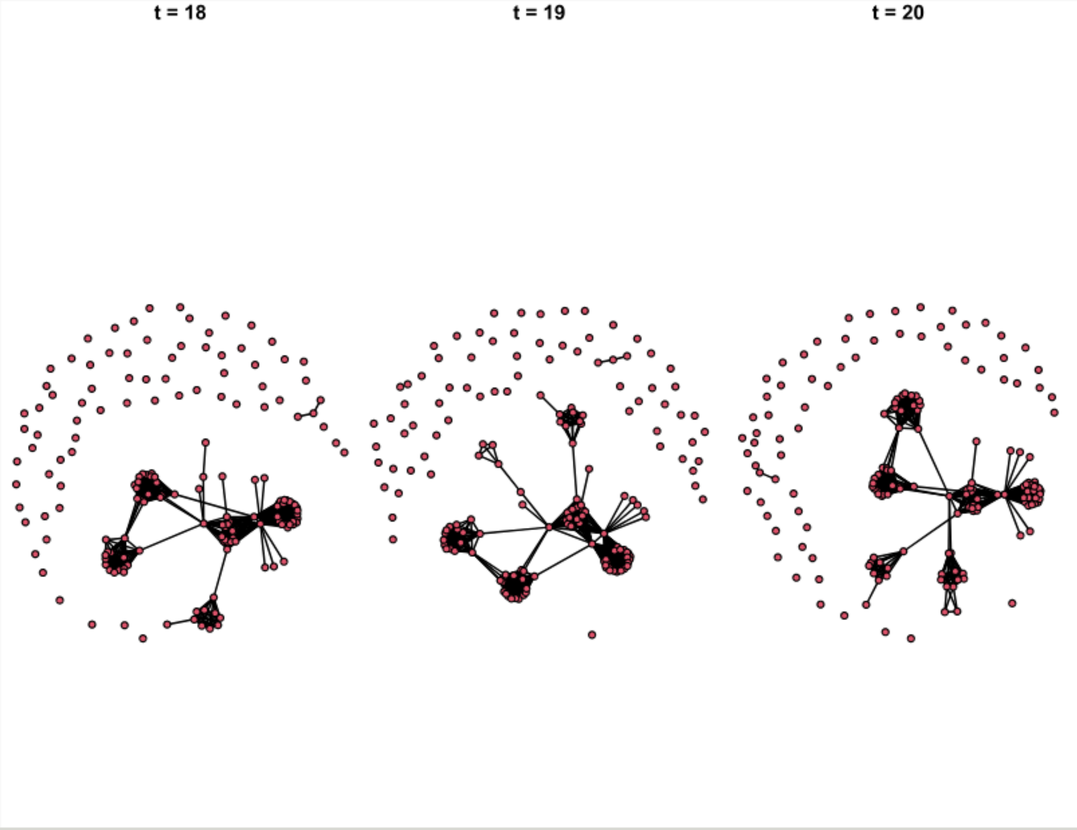
```
allianceNets
```

```
$1981
 Network attributes:
  vertices = 164
  directed = FALSE
  hyper = FALSE
  loops = FALSE
  multiple = FALSE
  bipartite = FALSE
  total edges= 625
    missing edges= 0
    non-missing edges= 625

 Vertex attribute names:
    cinc Composite_Index_National_Capability polity vertex.names year

No edge attributes

$1982
```

These are 20 networks, each as a `network` object, packaged together as a list. The first element is the observed network at $t = 1$ (1981), the second is the observed network at $t = 2$ (1982), etc.

Plotting the last three networks

```
par(mfrow = c(1, 3), mar = c(0, 0, 1, 0))
for (i in (length(allianceNets) - 2):length(allianceNets)) {
  plot(allianceNets[[i]], main = paste("t =", i))
}
```



How to include attributes

What	How to store
Constant dyadic covariates	Single network or matrix
Time-varying dyadic covariates	A list of networks or matrices
Node level attributes	As vertex attributes inside the observed network objects

Similar to what you are used to with ERGM's

How to include attributes

Name	What	Stored how?
Composite_Index_National_Capability	Node level attribute	vertex attribute
polity	Node level attribute	vertex attribute
year	Node level attribute	vertex attribute
sharedBorder	Constant dyadic covariate	matrix
lastYearsAdjacency	Time-varying dyadic covariate	list of 20 matrices
lastYearsSharedPartners	Time-varying dyadic covariate	list of 20 matrices
militaryDisputes	Time-varying dyadic covariate	list of 20 matrices

How to include attributes

Procedure | Application | The networks | **sharedBorder** | lastYearsAdjacency

lastYearsSharedPartners | militaryDisputes

```
class(sharedBorder)
```

```
[1] "matrix" "array"
```

```
dim(sharedBorder)
```

```
[1] 164 164
```

```
sharedBorder
```

```
    USA CAN CUB HAI DOM JAM TRI MEX GUA HON SAL NIC COS PAN COL VEN GUY ECU PER
USA   0   1   1   0   0   0   0   1   0   0   0   0   0   0   0   0   0   0   0
CAN   1   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
CUB   1   0   0   1   1   1   0   1   0   1   0   0   0   0   0   0   0   0   0
```

How to include attributes

Procedure | Application | The networks | sharedBorder | **lastYearsAdjacency**

lastYearsSharedPartners | militaryDisputes

```
class(lastYearsAdjacency)
```

```
[1] "list"
```

```
length(lastYearsAdjacency)
```

```
[1] 20
```

```
lastYearsAdjacency
```

```
$1981
    USA CAN CUB HAI DOM JAM TRI MEX GUA HON SAL NIC COS PAN COL VEN GUY ECU PER
USA   0   1   0   1   1   1   1   1   1   1   1   1   1   1   1   1   0   1   1
CAN   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
```

How to include attributes


```
class(lastYearsSharedPartners)
```

```
[1] "list"
```

```
length(lastYearsSharedPartners)
```

```
[1] 20
```

```
lastYearsSharedPartners
```

```
$1981
    USA CAN CUB HAI DOM JAM TRI MEX GUA HON SAL NIC COS PAN COL VEN GUY ECU PER
USA   0   0   0   0   1   2   3   4   5   6   7   8   9  10  11  12   0  13  14
CAN   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
```

How to include attributes


```
class(militaryDisputes)
```

```
[1] "list"
```

```
length(militaryDisputes)
```

```
[1] 20
```

```
militaryDisputes
```

```
$1981
    USA CAN CUB HAI DOM JAM TRI MEX GUA HON SAL NIC COS PAN COL VEN GUY ECU PER
USA   0   1   1   0   0   0   0   0   0   0   0   0   0   1   0   0   0   1   1
CAN   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0   0
```

First model

Model | Result | Interpretation | GoF (numbers)

```
p0 <- Sys.time()
model_1 <- btergm::btergm(allianceNets ~ edges +
                            gwesp(0, fixed = TRUE) +
                            edgecov(lastYearsSharedPartners) +
                            edgecov(militaryDisputes) +
                            nodecov("polity") +
                            nodecov("Composite_Index_National_Capability") +
                            absdiff("polity") +
                            absdiff("Composite_Index_National_Capability") +
                            edgecov(sharedBorder),
                          R = 100, # number of bootstraps
                          parallel = "snow", ncpus = 4  # optional line
                          )
Sys.time() - p0  # 41 secs on my computer
```

First model

SNA4DS

Model | Result | Interpretation | GoF (numbers)

```
btergm::summary(model_1)
```

```
                                                Estimate    2.5%   97.5%
edges                                         -5.0498497 -5.2282 -4.9030
gwesp.fixed.0                                  1.6242771  1.5476  1.7023
edgecov.lastYearsSharedPartners[[i]]           2.1065906  1.7966  2.4364
edgecov.militaryDisputes[[i]]                  0.2866346  0.1926  0.3786
nodecov.polity                                -0.0050764 -0.0109  0.0007
nodecov.Composite_Index_National_Capability    8.3308516  3.6427 12.4103
absdiff.polity                                -0.1257948 -0.1407 -0.1103
absdiff.Composite_Index_National_Capability   -4.8494093 -8.4005 -1.0835
edgecov.sharedBorder[[i]]                      3.2145679  3.1068  3.3218
```

First model

Model | Result | Interpretation | GoF (numbers)

	Estimate	2.5%	97.5%
edges	-5.0498497	-5.2282	-4.9030
gwesp.fixed.0	1.6242771	1.5476	1.7023
edgecov.lastYearsSharedPartners[[i]]	2.1065906	1.7966	2.4364
edgecov.militaryDisputes[[i]]	0.2866346	0.1926	0.3786
nodecov.polity	-0.0050764	-0.0109	0.0007
nodecov.Composite_Index_National_Capability	8.3308516	3.6427	12.4103
absdiff.polity	-0.1257948	-0.1407	-0.1103
absdiff.Composite_Index_National_Capability	-4.8494093	-8.4005	-1.0835
edgecov.sharedBorder[[i]]	3.2145679	3.1068	3.3218

Some results:

- There is a tendency towards "friend-of-a-friend"
- The more previous shared partners, the more likely is an alliance now
- Countries with higher *cinc* have more allies and countries prefer to ally with others with similar *cinc*
- Shared borders matter

First model

SNA4DS

Model | Result | Interpretation | GoF (numbers)

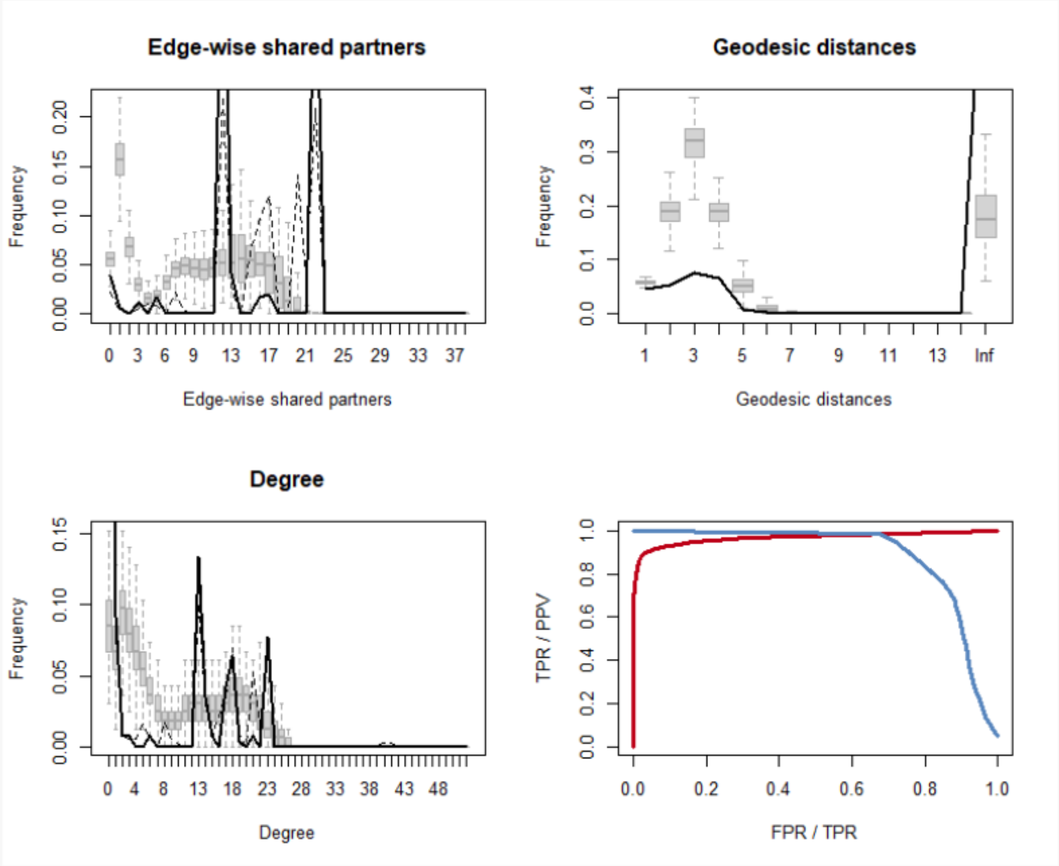
```
p0 <- Sys.time()
model_1_gof <- snafun::stat_plot_gof_as_btergm(model_1, silent = TRUE)
Sys.time() - p0  # 6min 40sec on my machine
```

>>> bunch of intermediate info skipped <<<

```
model_1_gof
```

```
   obs: mean median min max sim: mean median min max    Pr(>z)
0        29.7     28  24  40    85.670     86  40 148 < 2.2e-16 ***
1         5.7      4   0  12   241.500    242 124 362 < 2.2e-16 ***
2         1.4      0   0  12   106.126    106  30 204 < 2.2e-16 ***
3         7.8      8   0  28    46.102     46   6  98 9.427e-14 ***
4        13.5      0   0  30    25.723     24   0  64 0.0020606 **
5        12.0     12  12  12    29.968     28   0  78 < 2.2e-16 ***
6         0.0      0   0   0    49.826     50   8 132 < 2.2e-16 ***
7        28.8      0   0  72    68.505     70   8 120 9.797e-05 ***
8         4.3      0   0  86    74.231     76   4 140 9.518e-13 ***
9         0.1      0   0   2    71.477     72   4 156 < 2.2e-16 ***
10        0.1      0   0   2    70.537     68   4 166 < 2.2e-16 ***
```

gof plot for this model



Legend:

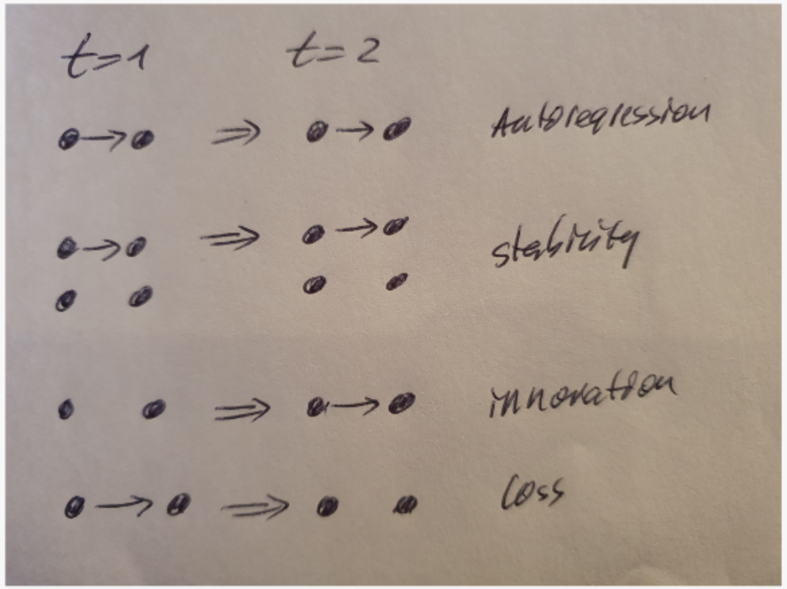
- grey boxes: gof simulations
- thick line: median from the 20 observed networks
- dashed line: mean from the 20 observed networks
- bottom right: **red** line is the ROC curve, the **blue** line the Precision-Recall curve.

That fit is terrible!!! Why?

How can we improve the model?

Time effects

Memory



- **positive autoregression** ▶ Previous existing edges persist in a next network
- **Dyadic stability** ▶ Both previous existing and non-existing ties are carried over to the current network
- **Edge innovation** ▶ A non-existing previous tie becomes existent in the current network
- **Edge loss** ▶ An existing previous tie is lost in the current network `

Time effects

Memory

- **positive autoregression** ▶ Previous existing edges persist in a next network

```
btergm::memory(type = "autoregression", lag = 1)
```

- **Dyadic stability** ▶ Both previous existing and non-existing ties are carried over to the current network

```
btergm::memory(type = "stability", lag = 1)
```

- **Edge innovation** ▶ A non-existing previous tie becomes existent in the current network

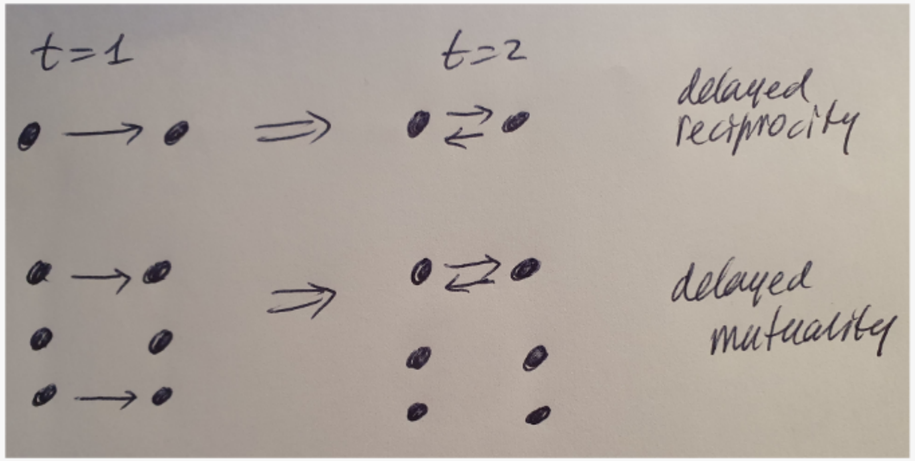
```
btergm::memory(type = "innovation", lag = 1)
```

- **Edge loss** ▶ An existing previous tie is lost in the current network

```
btergm::memory(type = "loss", lag = 1)
```

Time effects

Delayed reciprocity



- **reciprocity** ▶ if node *j* is tied to node *i* at *t* = 1, does this lead to a reciprocation of that tie back from *i* to *j* at *t* = 2?
- **mutuality** ▶ if node *j* is tied to node *i* at *t* = 1, does this lead to a reciprocation of that tie back from *i* to *j* at *t* = 2 **AND** if *i* is not tied to *j* at *t* = 1, will this lead to *j* not being tied to *i* at *t* = 2? This captures a trend *away from asymmetry*.

Time effects

Delayed reciprocity

- **reciprocity** ▶ if node *j* is tied to node *i* at *t* = 1, does this lead to a reciprocation of that tie back from *i* to *j* at *t* = 2?

```
btergm::delrecip(mutuality = FALSE, lag = 1)
```

- **mutuality** ▶ if node *j* is tied to node *i* at *t* = 1, does this lead to a reciprocation of that tie back from *i* to *j* at *t* = 2 **AND** if *i* is not tied to *j* at *t* = 1, will this lead to *j* not being tied to *i* at *t* = 2? This captures a trend *away from asymmetry*.

```
btergm::delrecip(mutuality = TRUE, lag = 1)
```

Time effects

Time covariates

- Test for a specific trend (linear or non-linear) for edge formation

  `btergm::timecov()` (for a linear trend)

  `btergm::timecov(transform = sqrt)` (for a geometrically decreasing trend)

- This can be combined with a covariate to create an interaction effect to test whether the importance of a covariate increases or decreases over time.

  `btergm::timecov(militaryDisputes)` (for a linear trend of `militaryDisputes` over time)

  `btergm::timecov(militaryDisputes, transform = sqrt)`

Model 2

SNA4DS

Model | Result | Interpretation | GoF (numbers)

```
p0 <- Sys.time()
model_2 <- btergm::btergm(allianceNets ~ edges +
                    gwesp(0, fixed = TRUE) +
                    edgecov(lastYearsSharedPartners) +
                    edgecov(militaryDisputes) +
                    nodecov("polity") +
                    nodecov("Composite_Index_National_Capability") +
                    absdiff("polity") +
                    absdiff("Composite_Index_National_Capability") +
                    edgecov(sharedBorder) +
                    memory(type = "stability") +
                    timecov() +
                    timecov(militaryDisputes),
                  R = 100
                  , parallel = "snow", ncpus = 16  # optional line
                  )
Sys.time() - p0   # 167 secs on my machine
```

Model 2

Model | Result | Interpretation | GoF (numbers)

```
btergm::summary(model_2)
```

	Estimate	2.5%	97.5%
edges	-3.139220	-3.7956	-2.4224
gwesp.fixed.0	1.790187	1.5433	1.9951
edgecov.lastYearsSharedPartners[[i]]	0.113878	0.0614	0.4283
edgecov.militaryDisputes[[i]]	-1.137333	-2.4417	0.2991
nodecov.polity	-0.015363	-0.0572	0.0197
nodecov.Composite_Index_National_Capability	-0.163806	-15.0743	13.2839
absdiff.polity	-0.068772	-0.1251	-0.0104
absdiff.Composite_Index_National_Capability	3.274009	-9.2209	14.4605
edgecov.sharedBorder[[i]]	1.597682	0.5699	2.6708
edgecov.memory[[i]]	5.097553	4.6231	6.2539
edgecov.timecov1[[i]]	0.072603	0.0228	0.1637
edgecov.timecov2.militaryDisputes[[i]]	0.054838	-0.0608	0.1193

Model 2

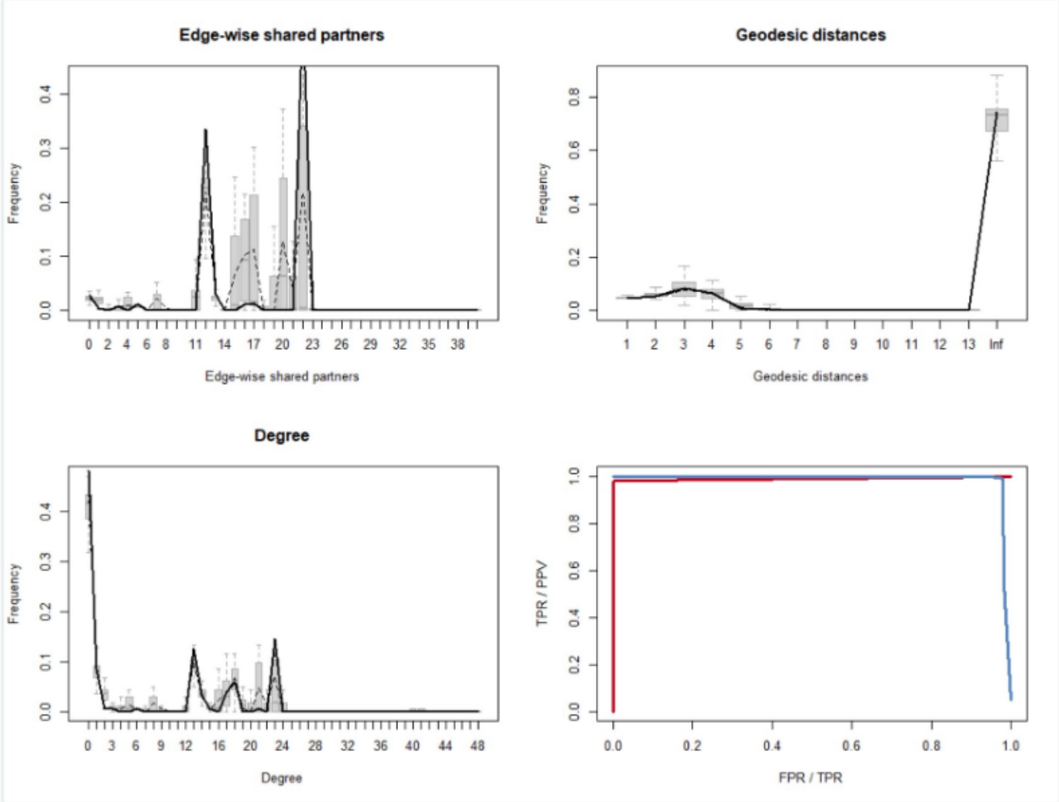
Model | Result | **Interpretation** | GoF (numbers)

	Estimate	2.5%	97.5%
edges	-3.139220	-3.7956	-2.4224
gwesp.fixed.0	1.790187	1.5433	1.9951
edgecov.lastYearsSharedPartners[[i]]	0.113878	0.0614	0.4283
edgecov.militaryDisputes[[i]]	-1.137333	-2.4417	0.2991
nodecov.polity	-0.015363	-0.0572	0.0197
nodecov.Composite_Index_National_Capability	-0.163806	-15.0743	13.2839
absdiff.polity	-0.068772	-0.1251	-0.0104
absdiff.Composite_Index_National_Capability	3.274009	-9.2209	14.4605
edgecov.sharedBorder[[i]]	1.597682	0.5699	2.6708
edgecov.memory[[i]]	5.097553	4.6231	6.2539
edgecov.timecov1[[i]]	0.072603	0.0228	0.1637
edgecov.timecov2.militaryDisputes[[i]]	0.054838	-0.0608	0.1193

Some results:

- There is a tendency towards "friend-of-a-friend"
- Shared borders matter
- There is a preference for stability over time (alliances tend not to start or dissolve much)
- There is a slightly positive trend in creating alliances

gof plot for this model



Legend:

- grey boxes: simulations
- thick line: median from the 20 observed networks
- dashed line: mean from the 20 observed networks

That fit is wonderful!